

University of Central Punjab

BSCS FINAL PROJECT
Software Design Specification

**AI-Powered Navigator for Ethical Obscure Net Exploration and
Intelligence Gathering**



Project Advisor

Hafiz Bilal Shahid

Presented by:

Group ID: F25CS064

Student Reg#
L1F22BSCS0004
L1F22BSCS0028
L1F22BSCS0032

Student Name
Ali Naeem
M. Omar Zahid
M. Abdullah

Faculty of Information Technology & Computer Science
University of Central Punjab

Software Design Specification

Version 1

AI-Powered Navigator for Ethical Obscure Net Exploration and Intelligence Gathering

Advisor: Hafiz Bilal Shahid

Group ID: F25CS064

Member Name	Primary Responsibility
Ali Naeem	Database, Research and Testing
M. Omar Zahid	Frontend Development and Dashboard Design
M. Abdullah	Backend Development and AI Integration

Table of Contents

Table of Contents	i
Revision History	ii
Previous Phases Feedback.....	ii
Abstract.....	iii
1. Introduction.....	1
1.1 Product.....	1
1.2 Background.....	1
1.3 Objective(s)/Aim(s)/Target(s)	1
1.4 Scope	1
1.5 Business Goals.....	2
1.6 Document Conventions	2
1.7 Miscellaneous	2
2. Overall Description	3
2.1 Product Features	3
2.2 Functional Description	3
2.3 User Classes and Characteristics	3
2.4 Design and Implementation Constraints.....	4
2.5 Assumptions and Dependencies	4
3. Technical Architecture	5
3.1 Application and Data Architecture	7
3.2 Component Interactions and Collaborations	12
3.3 Design Reuse and Design Patterns	15
3.4 Technology Architecture	15
4. Screenshots/Prototype	16
4.1 Workflow.....	16
4.2 Screens.....	17
4.3 Additional Information	21
5. Other Design Details	21
6. Revised Project Plan	22
7. References	22
Appendix A: Glossary.....	23
Appendix B: IV & V Report	24

Revision History

Name	Date	Reason For Changes	Version

Previous Phases Feedback

Idea Defence Feedback (Screenshot)

Dear Students,

Following is the feedback regarding your Idea Defence:

Sr. #	CS-167
Project Title	AI-POWERED NAVIGATOR FOR ETHICAL OBSCURE NET EXPLORATION AND INTELLIGENCE GATHERING

Confused about AI model. Redefine scope.

They don't know about the implementation.

Major revision required. Revise your project idea in light of comments provided. Last date to submit the revised idea form is **Thursday, September 11, 2025, before 3:00 PM** in Project Office (F-304 Building A).

Abstract

This project presents the development of an AI-driven system designed to ethically explore and analyze content from obscure net (.onion) sources for cybersecurity intelligence purposes. The obscure net contains valuable information relating to leaked data, cybercrime activities, emerging threats, and other cybersecurity-relevant material, but accessing this information is inherently slow, risky, and technically challenging. The proposed system leverages Tor-based crawling, Natural Language Processing (NLP), and machine learning to automatically collect, classify, and filter obscure net content. A hybrid model that combines BERT embeddings with classical machine learning classifiers, such as Support Vector Machines (SVM), Random Forests, and Logistic Regression, will be employed to detect sensitive information, leaked records, and threat-related material. Ethical safeguards, role-based access, and content filtering mechanisms are integrated to ensure compliance with legal and moral standards. The final product is a secure, web-based platform that enhances the efficiency, accuracy, and safety of intelligence gathering from obscure net sources.

[1] Introduction

1.1 Product

The system under development is a secure, AI-based platform aimed at automating intelligence gathering from selected obscure net (.onion) websites. Its primary function is to crawl these websites, analyze the content, and identify potential threats, leaked credentials, and other sensitive information. Unlike traditional manual exploration, which is slow, error-prone, and exposes users to illegal or harmful content, this platform provides a controlled environment that ensures ethical and legal use. The product is designed specifically for cybersecurity teams, researchers, and agencies who require structured and actionable insights from otherwise unstructured and hidden online sources.

1.2 Background

The obscure net hosts vast volumes of unindexed, unstructured information that can be highly relevant to cybersecurity, law enforcement, and research purposes. This content is often used by cybercriminals to share data, conduct illicit transactions, or discuss illegal activities anonymously. Manual exploration of such sources is not only technically demanding but also exposes the user to significant risk, including malware, phishing attempts, and illegal material. Existing tools, such as dark web search engines or crawlers, rely primarily on keyword-based search and fail to provide semantic understanding or automated classification of content. There is a clear and pressing need for an intelligent, ethical system capable of safely analyzing obscure net content in a structured and meaningful way.

1.3 Objective(s)/Aim(s)/Target(s)

- Develop an ethical obscure net intelligence tool
- Automate crawling, extraction and processing of .onion content
- Identify leaked credentials, sensitive data and threat indicators
- Provide real-time classification through hybrid ML models
- Ensure security, access control and compliance
- Create a user-friendly dashboard for searched results
- Reduce manual analysis effort and increase detection efficiency

1.4 Scope

The scope of the project includes multiple key components. First, a Tor-based crawler will be developed to navigate and extract content from a curated list of .onion websites. The extracted data will undergo preprocessing and cleaning using NLP techniques to remove noise and structure the text for further analysis. Next, the textual content will be converted into semantic vector representations using BERT embeddings, enabling the system to understand the contextual meaning of the text. A hybrid classification pipeline, combining BERT embeddings with classical machine learning algorithms such as SVM, Random Forest, and Logistic Regression, will categorize the data into potential leaks, sensitive information, and threat indicators. Users will interact with the system through a secure web dashboard that allows search, filtering, and visualization of classified results. Access will be managed using role-based controls, and all activity will be logged to ensure accountability. The system is explicitly designed not to facilitate unrestricted browsing or any illegal activity.

1.5 Business Goals

- Provide organizations with early detection of leaks
- Improve cybersecurity monitoring capabilities
- Reduce cost and time required for threat intelligence
- Provide a research tool for academic and investigative use
- Establish a scalable foundation for advanced cybersecurity solutions

1.6 Document Conventions

Standards or typographical conventions that were followed when writing this SRS, such as fonts or highlighting that have special significance. The document convention is given below:

1. **Headings**

Headings and subheadings are often styled consistently for easy navigation. instance, sections and subsections are in bold.

2. **Font style and Family**

There is different type of font style but in this SRS, Bold is only use for main heading and subheadings remaining text are in simple format with Times new roman font style and the size of the text is 12.

3. **Bullet point and numbering**

Bullets and numbering are used for ordered lists. This helps organize information and improve readability.

4. **References**

References are given in specific IEEE format.

1.7 Miscellaneous

The project strictly adheres to ethical guidelines by operating in a passive, read-only mode that filters out illegal binaries (images/videos). Development follows GitFlow and PEP 8 standards to ensure code quality, while operational security is maintained through Docker containerization to prevent attribution. Additionally, an emergency "Kill Switch" protocol is defined to sever connections instantly, and daily database backups ensure data recovery.

[2] Overall Description

2.1 Product Features

The key features of the system include:

- **Tor-enabled Crawler:** A crawler capable of safely accessing onion domains, extracting textual content while handling Tor-specific latency and connection instability.
- **NLP Preprocessing Pipeline:** Cleaning, tokenization, lemmatization, and multilingual support to standardize text for AI analysis.
- **Semantic Embeddings with BERT:** Generating contextual vector representations for each text snippet to capture nuanced meaning.
- **Hybrid Machine Learning Classification:** Combining BERT embeddings with SVM, Random Forest, and Logistic Regression models for multi-class categorization.
- **Leak and Threat Detection Module:** Identifies sensitive information, leaked credentials, and cybersecurity threats.
- **Role-Based Web Dashboard:** Interactive interface for search, filtering, visualization, and reporting.
- **Activity Logging:** Detailed logs for auditing user actions, crawler access, and AI decisions.
- **Secure Database Storage:** Encrypted storage for all crawled data and AI-generated classifications, ensuring confidentiality and integrity.

2.2 Functional Description

The AI-Powered Navigator automates the discovery and classification of Dark Web content to identify potential security threats. It functions by routing a secure crawler through the Tor network to fetch onion sites, which are then processed by a BERT-based AI engine to detect leaks or threats. The final intelligence is presented on a searchable dashboard, enabling analysts to monitor risks without direct exposure to malicious networks.

2.3 User Classes and Characteristics

- **Admin:** Full control over crawler configuration, user management, and system monitoring. Must understand cybersecurity and ethical guidelines.
- **Cybersecurity Analyst:** Access to classified results and dashboards for threat analysis. Intermediate technical knowledge is sufficient.
- **Researcher/Journalist:** Non-technical users can search and filter results to access insights without interacting with raw data or crawling operations.

2.4 Design and Implementation Constraints

The system faces several constraints:

- Tor network latency and frequent downtime may affect crawling performance.
- Legal and ethical restrictions limit which domains can be accessed.
- Hardware limitations can affect BERT embedding generation due to high memory and computational requirements.
- Only text-based content is stored; downloading images, videos, or executable files is prohibited.
- Strong encryption and role-based access must be implemented for all sensitive data.

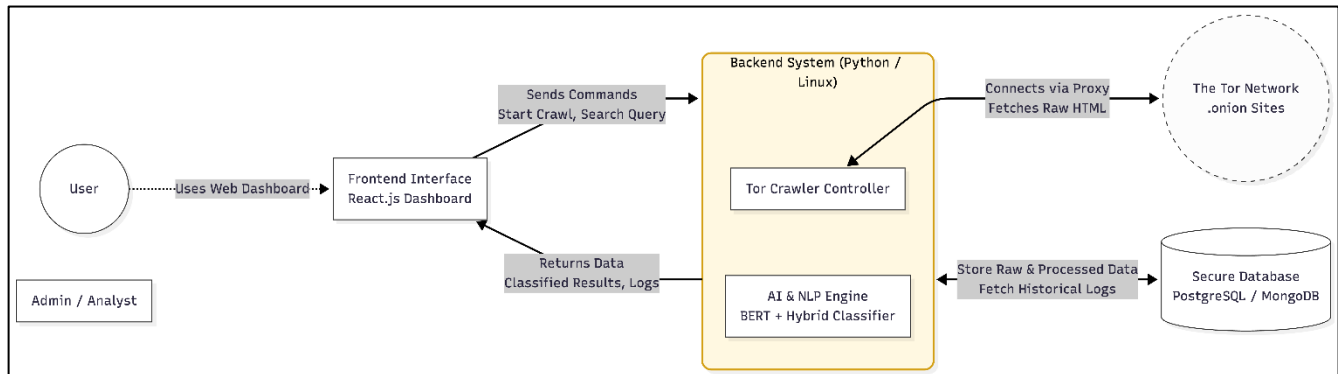
2.5 Assumptions and Dependencies

The system assumes:

- The Tor network remains operational and accessible.
- Selected .onion domains are active and accessible.
- Sufficient and high-quality training data for ML model development is available.
- Users follow ethical guidelines and access controls.
- Python libraries, AI frameworks, and server dependencies function correctly without conflicts.

[3] Technical Architecture

< Identify and describe the technical architecture of the current system. I



The system follows a **Layered Client-Server Architecture**, ensuring a clear separation of concerns between the user interface, business logic, and data storage. This modular design supports scalability and secure isolation of the crawling mechanism.

- **Presentation Layer (Client):** A browser-based **Thin Client** developed in **React.js**. It communicates with the backend via RESTful APIs and is responsible solely for data visualization and user input.
- **Application Layer (Server):** The core logic is built using **Python**. It hosts the Crawler Controller, Authentication Service, and the AI Engine (NLP + BERT). This layer acts as the bridge, processing requests from the client and managing complex interactions with the Tor network.
- **Data Layer:** A centralized relational database manages persistent storage for user credentials, raw crawled text, and classified intelligence.

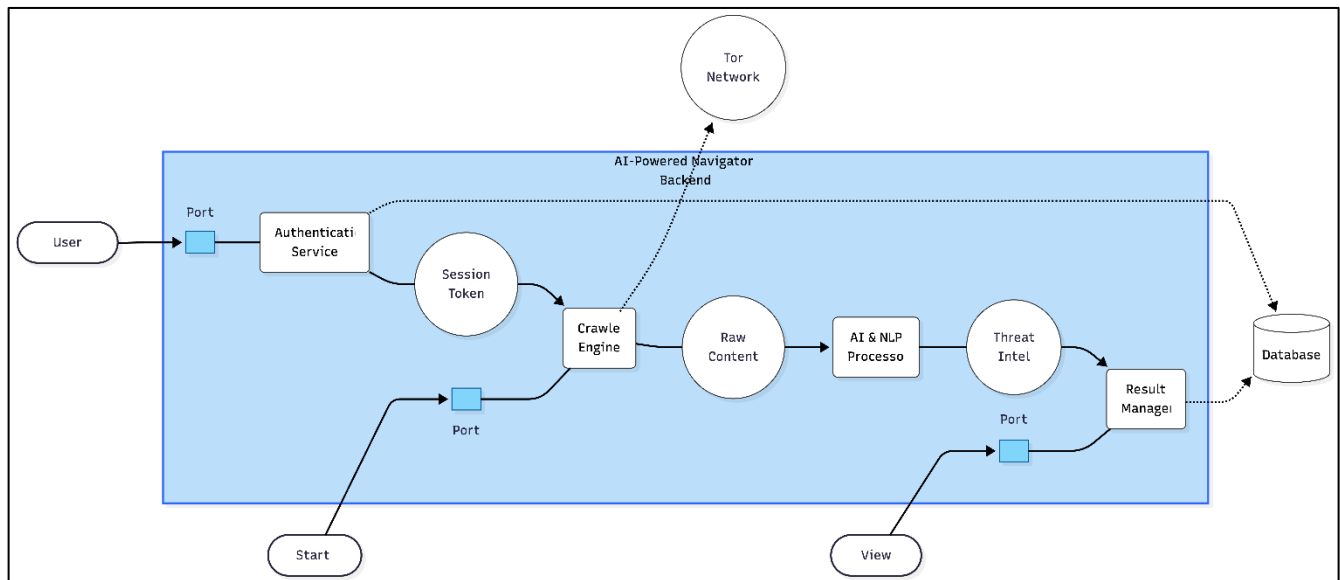
The system collects and manages three distinct categories of data:

1. **System Data:** Encrypted user credentials, API tokens, and crawler configuration settings.
2. **Raw Input Data:** Unstructured HTML and text content scraped from .onion sites.
3. **Analytical Data:** Structured records containing threat scores, classification tags, discovery timestamps, and source URLs.

Component	Technology	Description
Programming Language	Python 3.x	Primary language for backend logic, crawling, and AI integration.
Frontend Framework	React.js	JavaScript library for building the interactive user dashboard.
Network Protocol	Tor (SOCKS5)	Utilized for anonymized routing to access.onion hidden services.
AI/ML Engine	BERT / TensorFlow	Pre-trained models used for vectorization and text classification.
Database Platform	PostgreSQL	Relational database for structured storage of user data and analysis results.
Operating System	Linux (Ubuntu)	Preferred server environment for Tor compatibility and Python performance.
Web Server	Nginx / Gunicorn	Serves the application and manages API requests.

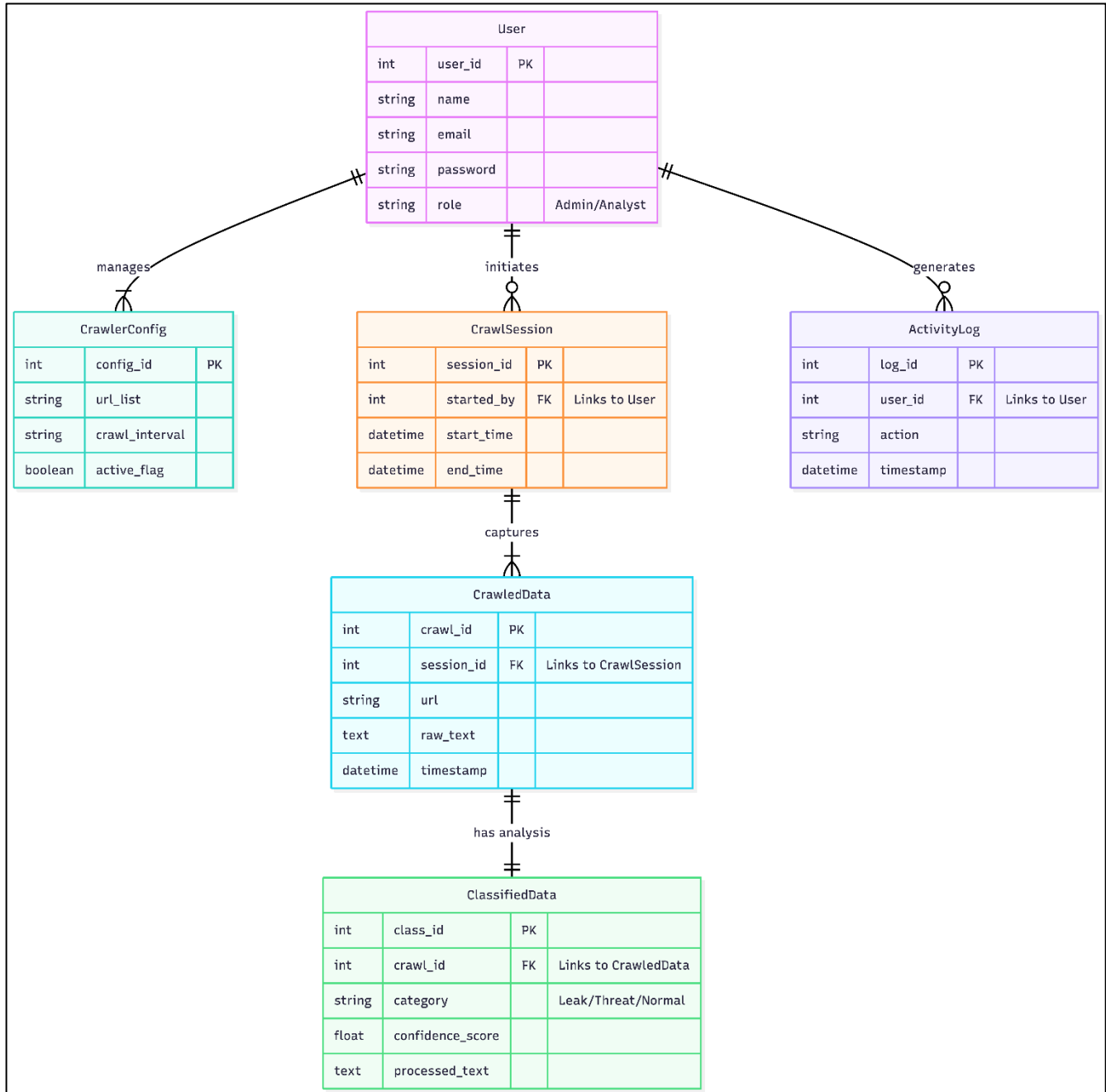
3.1 Application and Data Architecture

Component Diagram



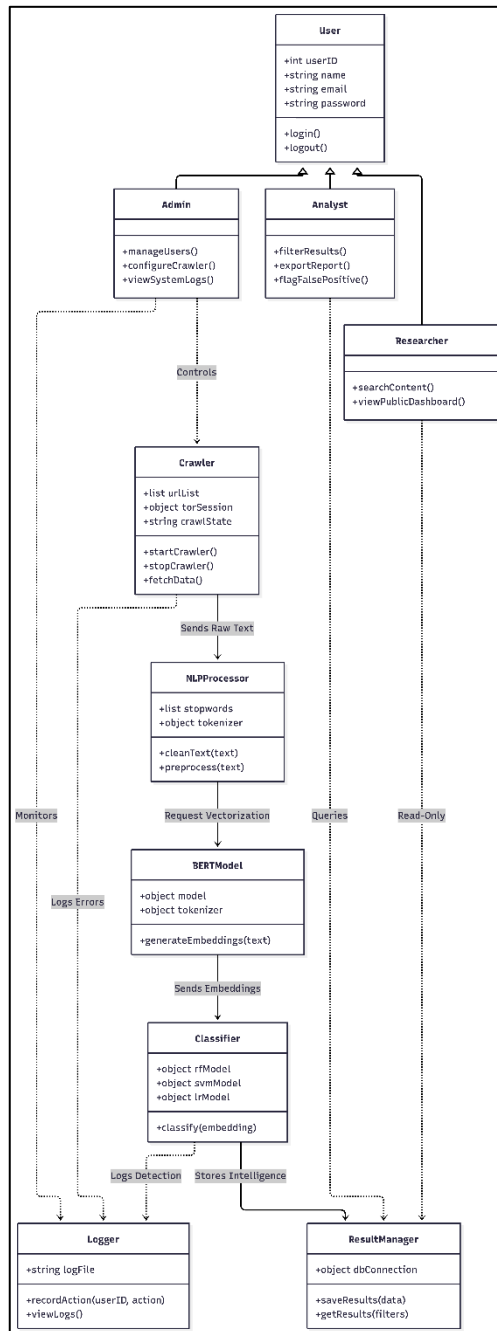
This structural diagram breaks down the backend system into modular components and defines their interfaces using "Ball-and-Socket" notation. It visualizes the **Authentication Service**, **Crawler Engine**, **AI Processor**, and **Result Manager** as distinct units encapsulated within the backend system. The diagram highlights how these components communicate: the Crawler provides raw data that the AI Processor requires, and the AI Processor provides intelligence that the Result Manager requires for storage. External ports on the left show system inputs (Credentials, Commands), while dependencies on the right show connections to the Tor Network and Database.

ERD



This diagram defines the logical structure of the system's database, focusing on data integrity and relationships. It centers on the **User** entity, which manages **CrawlerConfigs** and initiates **CrawlSessions**. The schema enforces a clear separation between raw and processed data: a **CrawlSession** captures multiple **CrawledData** entries (raw text), which are linked one-to-one with **ClassifiedData** entries (AI results). This design ensures that raw evidence is preserved independently of its classification. Additionally, an **ActivityLog** table is linked to the **User** to maintain a secure audit trail of all actions.

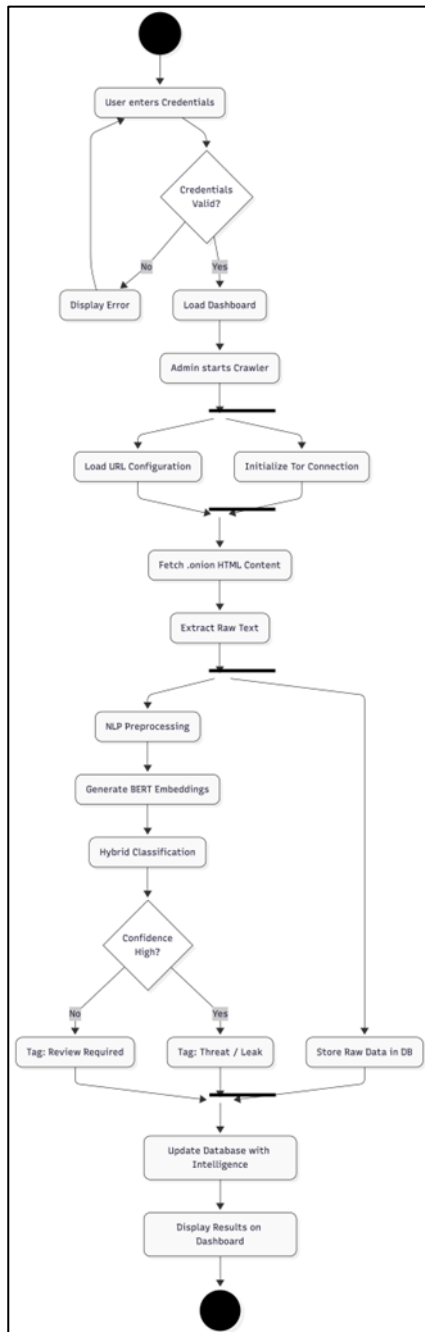
Class Diagram



This diagram models the static structure of the system's software classes. It uses namespaced groups to organize classes into logical domains: **User Management**, **Backend Core**, and **AI Pipeline**. Key features include the inheritance hierarchy for users (Admin, Analyst, Researcher) and the composition of the AI engine (NLPProcessor, BERTModel, Classifier). It defines the specific attributes and methods for each class, such as the

cleanText() method in the NLP module or the configureCrawler() method for Admins, providing a blueprint for development.

Activity Diagram

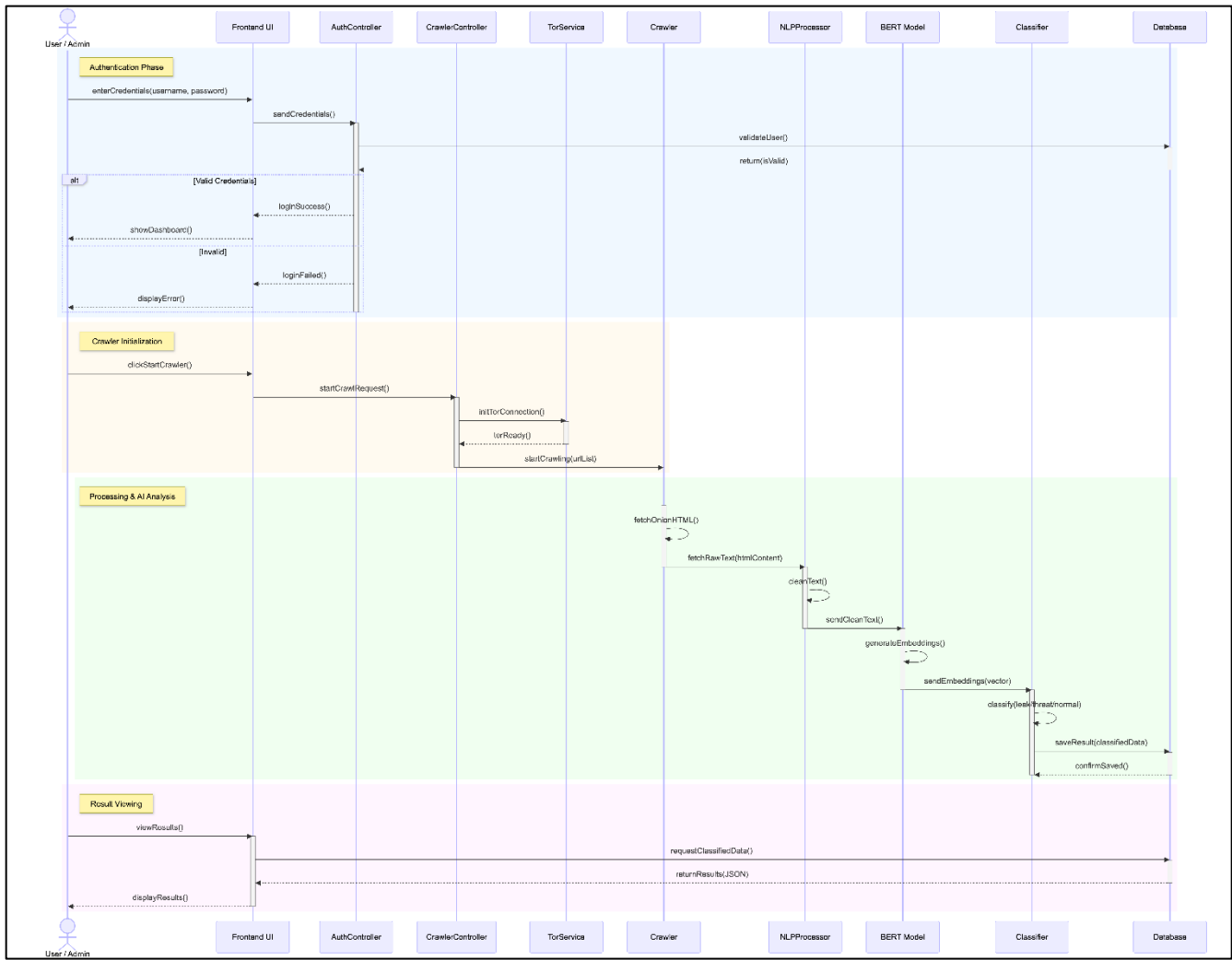


This UML Activity Diagram visualizes the dynamic, sequential behavior of the system from start to finish. It begins with the **Authentication Phase**, where the system verifies user credentials before granting access. The workflow then splits into parallel tasks using synchronization bars (forks): while the **Crawler** initializes the Tor connection and fetches .onion content, the system prepares for **AI Processing**. A second synchronization point occurs after data extraction, where raw text is simultaneously archived in the database and processed by the NLP/BERT models. The diagram concludes with the final classification of data and its display on the results dashboard.

Decision Table

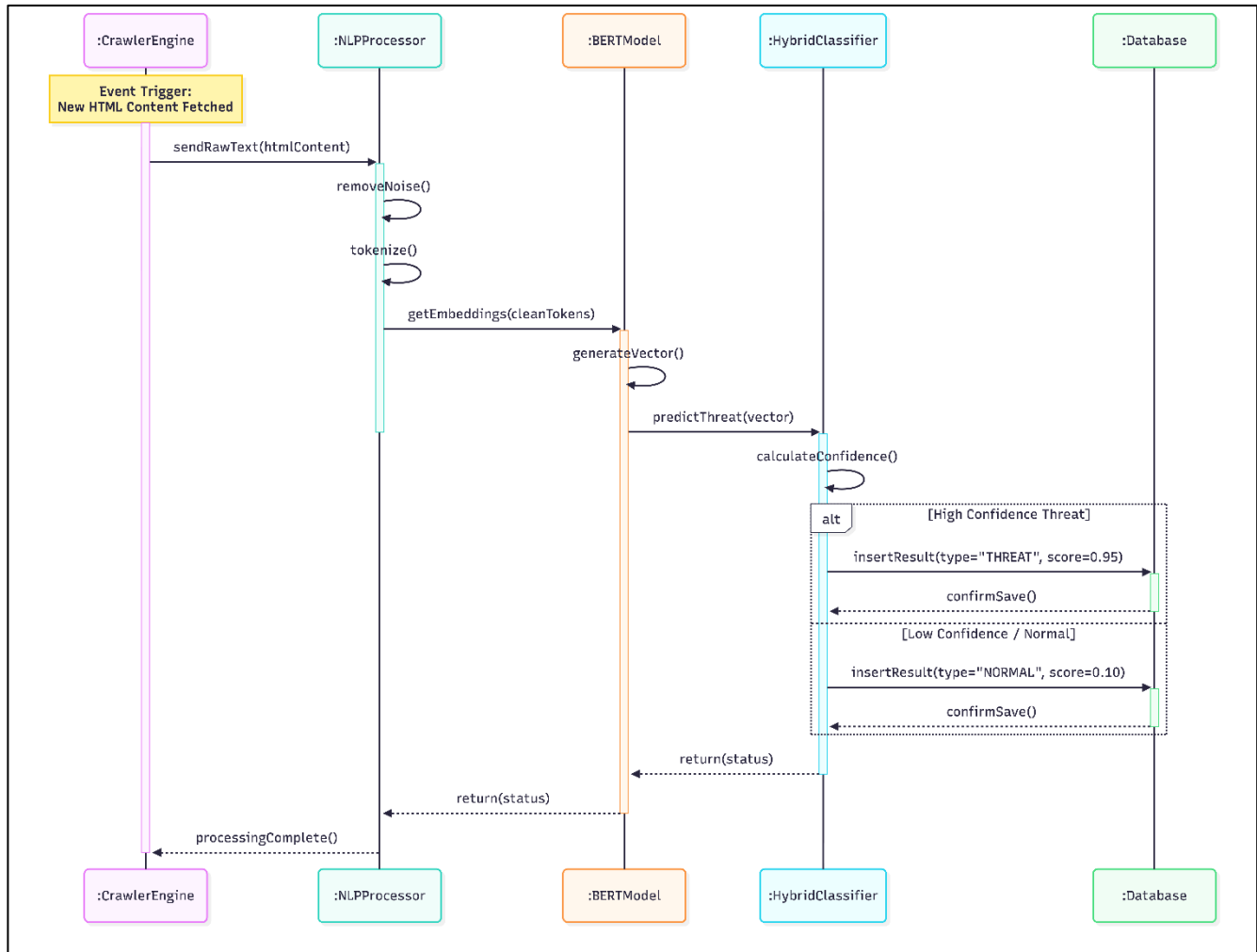
Condition /Action	Leak	Threat	Normal	Low Confidence	Illegal
Leak keywords	Y	N	N	N	N
Threat keywords	N	Y	N	N	N
Normal text	N	N	Y	N	N
Confidence<Threshold	N	N	N	Y	N
Illegal content	N	N	N	N	Y
Action	Leak	Threat	Normal	Uncertain	Block

3.2 Component Interactions and Collaborations



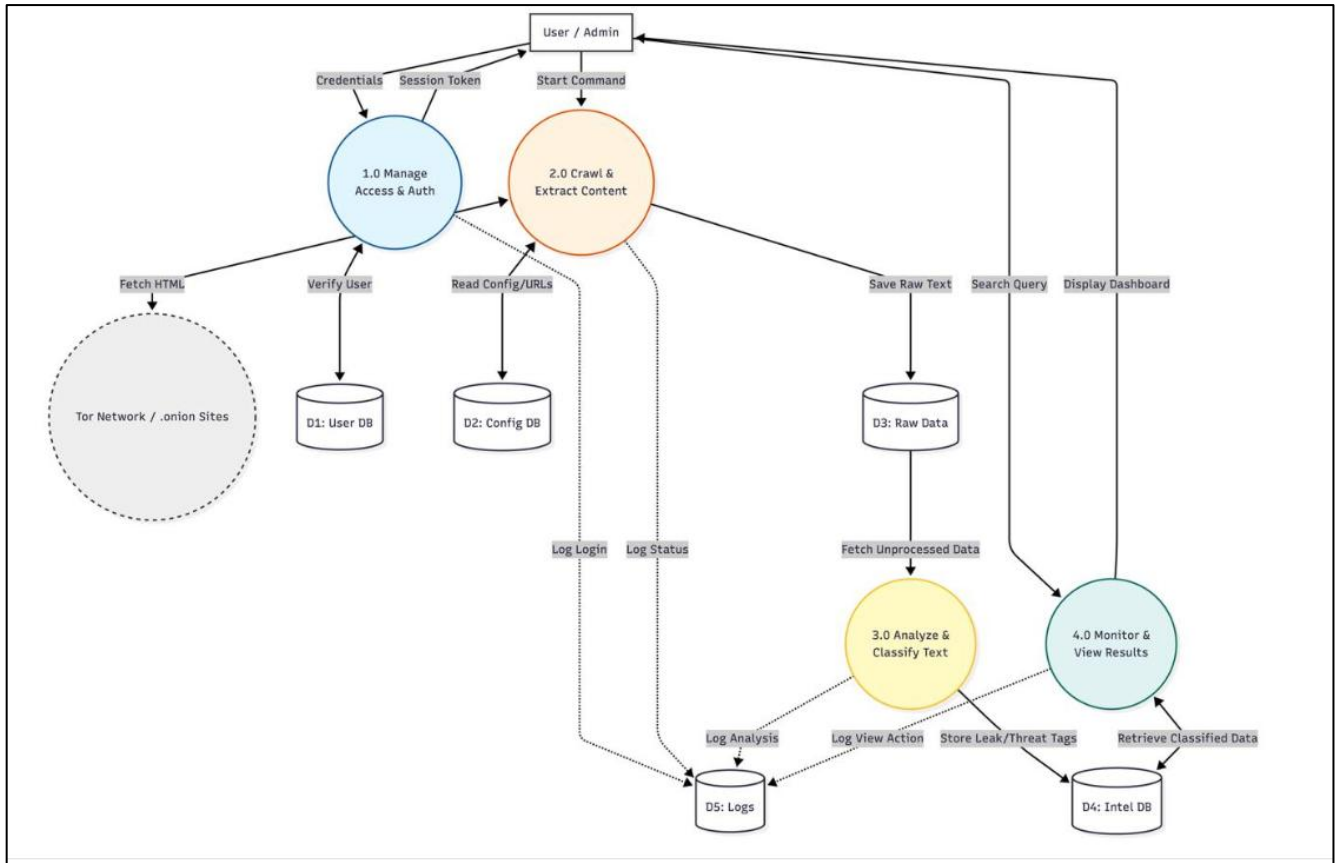
This diagram plots the timeline of interactions across the system's lifecycle. It is divided into four distinct phases: **Authentication**, **Initialization**, **Processing**, and **Visualization**. Vertical lifelines represent objects (like the Crawler, BERT Model, and Database), while horizontal arrows show the specific messages exchanged (e.g., `initTorConnection()`, `generateEmbeddings()`). This diagram is crucial for understanding the chronological order of operations, specifically how the system handles the latency of Tor connections and the sequential pipeline of AI processing.

Event Trace for Single Page Analysis



This diagram details the specific execution trace for processing a single .onion webpage. The event is triggered when the **Crawler Engine** successfully retrieves raw HTML content. The trace shows the strict dependency chain where data is first sanitized by the **NLPProcessor**, converted into numerical vectors by the **BERT Model**, and finally evaluated by the **HybridClassifier**. It highlights the conditional logic where the Classifier determines the “Threat Score” before committing the final intelligence record to the **Database**, ensuring only processed, structured data is stored.

DFD



This diagram provides a high-level view of the system's data processing. It identifies four main processes: **Manage Access**, **Crawl Content**, **Analyze Text**, and **View Results**. It shows the primary data flow between these processes and the five main data stores (User DB, Config DB, Raw Data, Intel DB, Logs). This level is useful for understanding the broad strokes of data movement, such as how user commands trigger crawling and how the resulting data eventually feeds back into the dashboard for viewing.

3.3 Design Reuse and Design Patterns

The application architecture implements several standard design patterns to ensure maintainability and scalability:

- **Singleton Pattern:** Applied to the **Tor Proxy Controller** and **Database Connection Pool**. This ensures that the system maintains only one active connection manager for the Tor network (preventing port conflicts) and efficiently manages database handles.
- **Model-View-Controller (MVC):** The backend follows the MVC (or MVT in Python/Django) pattern to strictly separate the **Data Models** (Database Tables), the **Controller Logic** (Crawler and AI processing), and the **View** (JSON API responses).
- **Strategy Pattern:** Used in the **Analysis Engine** to switch between different processing strategies at runtime. For example, the system can dynamically switch between a "Fast Heuristic Scan" (keyword matching) and a "Deep Neural Analysis" (BERT) depending on the system load or the perceived risk of the content.

3.4 Technology Architecture

System Hosting

- **Private Infrastructure:** Unlike typical web apps, this system **cannot** be hosted on public shared Platform-as-a-Service (PaaS) providers (like Heroku or standard AWS Beanstalk) due to the sensitivity of the data and the need for non-standard outbound ports (Tor).
- **Deployment Model:** The system will be hosted on a **Private Secure VPS** or an **On-Premise Enterprise Server**. The application components (Crawler, Database, Web Server) will be containerized using **Docker** to ensure isolation from the host operating system.

Connectivity Requirements

- **External (Outbound):** The architecture requires a specialized network configuration where **all** outbound traffic is blocked by the firewall except traffic routed through localhost:9050 (The Tor SOCKS5 Proxy). This ensures no accidental IP leaks occur.
- **Internal (Inbound):** Access to the web interface is restricted to the **Internal LAN** or a **Private VPN**. No inbound ports will be exposed to the public internet.

Modes of Operation

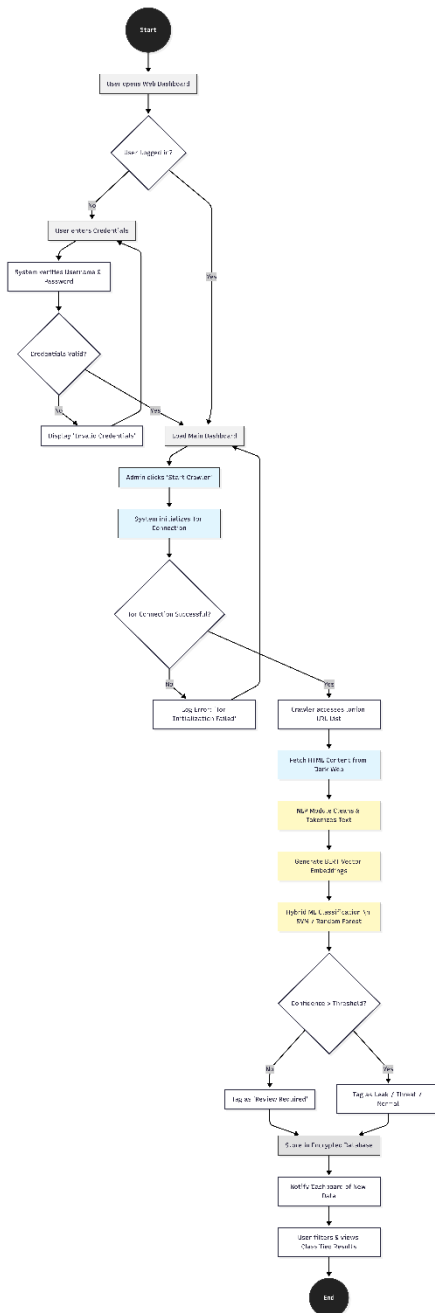
The architecture supports two concurrent modes of operation:

1. **Daemon Mode (Background):** The **Crawler** and **Scheduler** run as headless background services (systemd daemons), continuously fetching and processing data without user interaction.

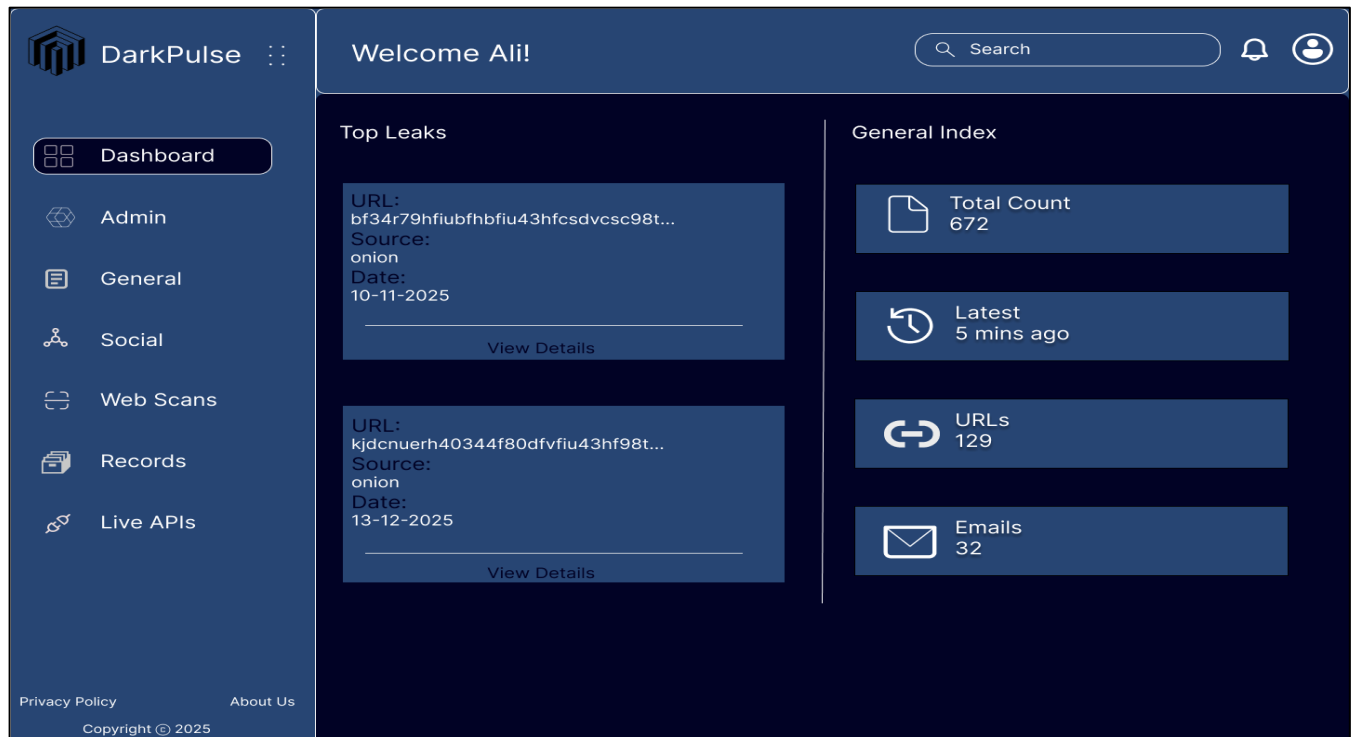
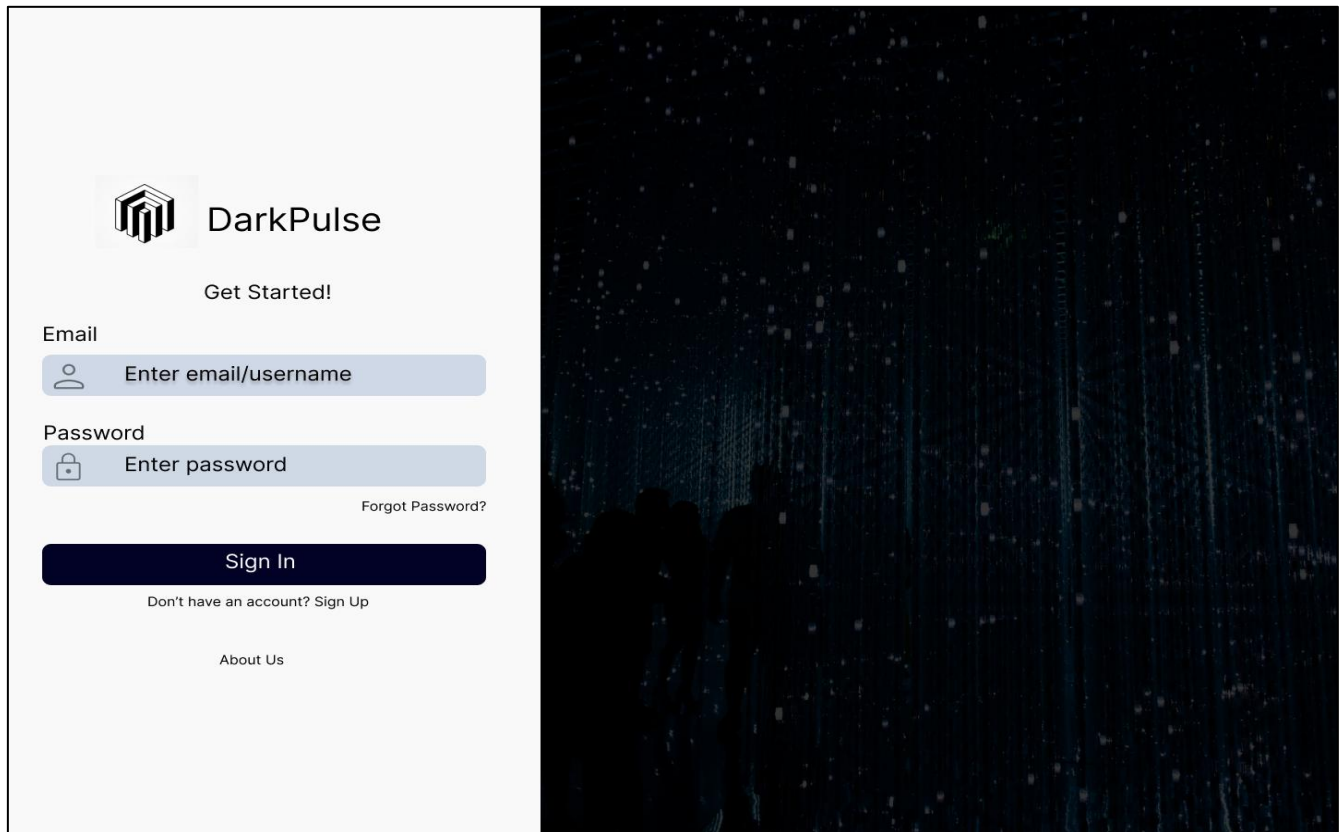
2. **Interactive Mode (Foreground):** The **Web Server** operates in real-time to serve API requests for the dashboard, allowing users to query the database populated by the background daemons.

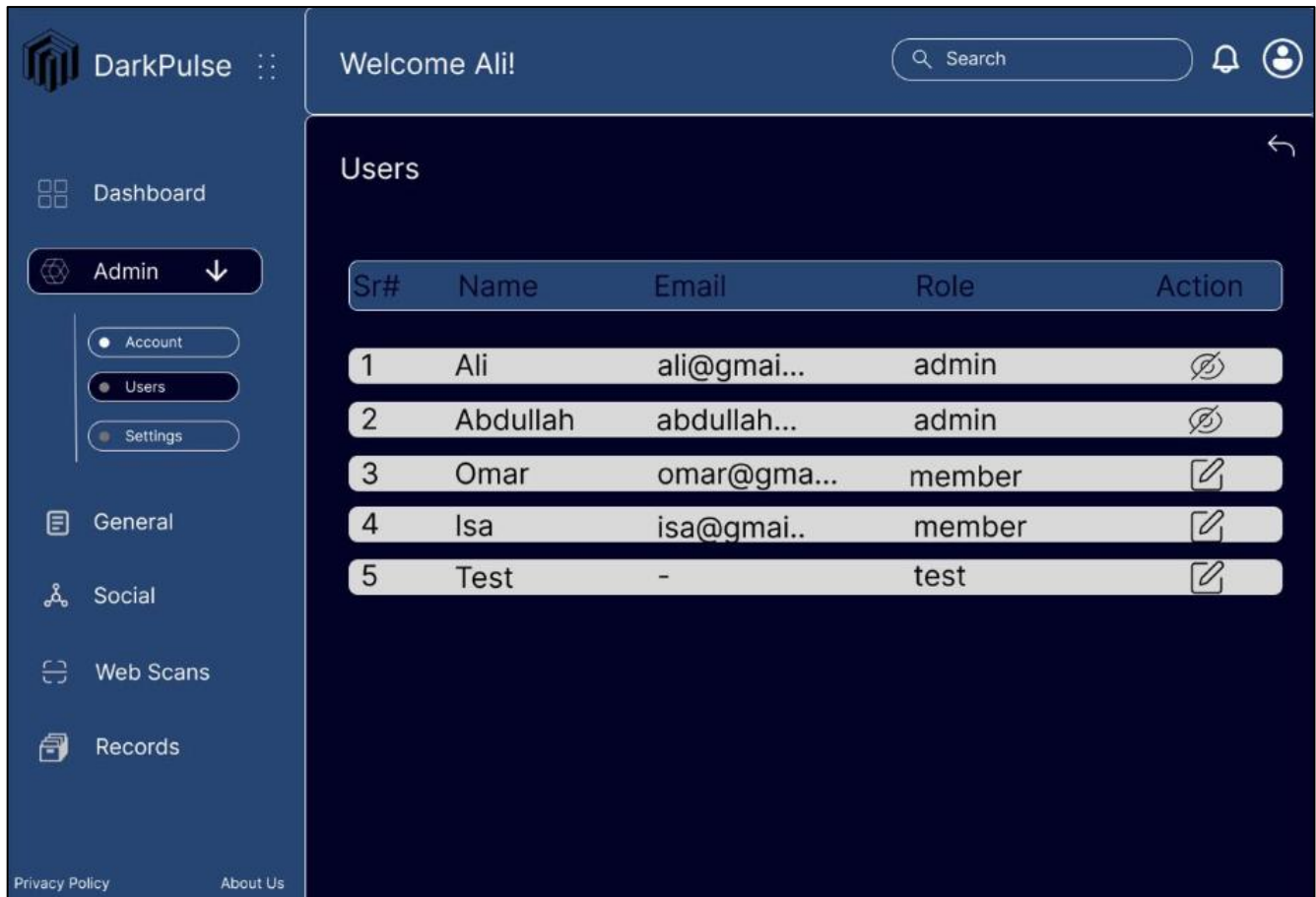
[4] Screenshots/Prototype

4.1 Workflow



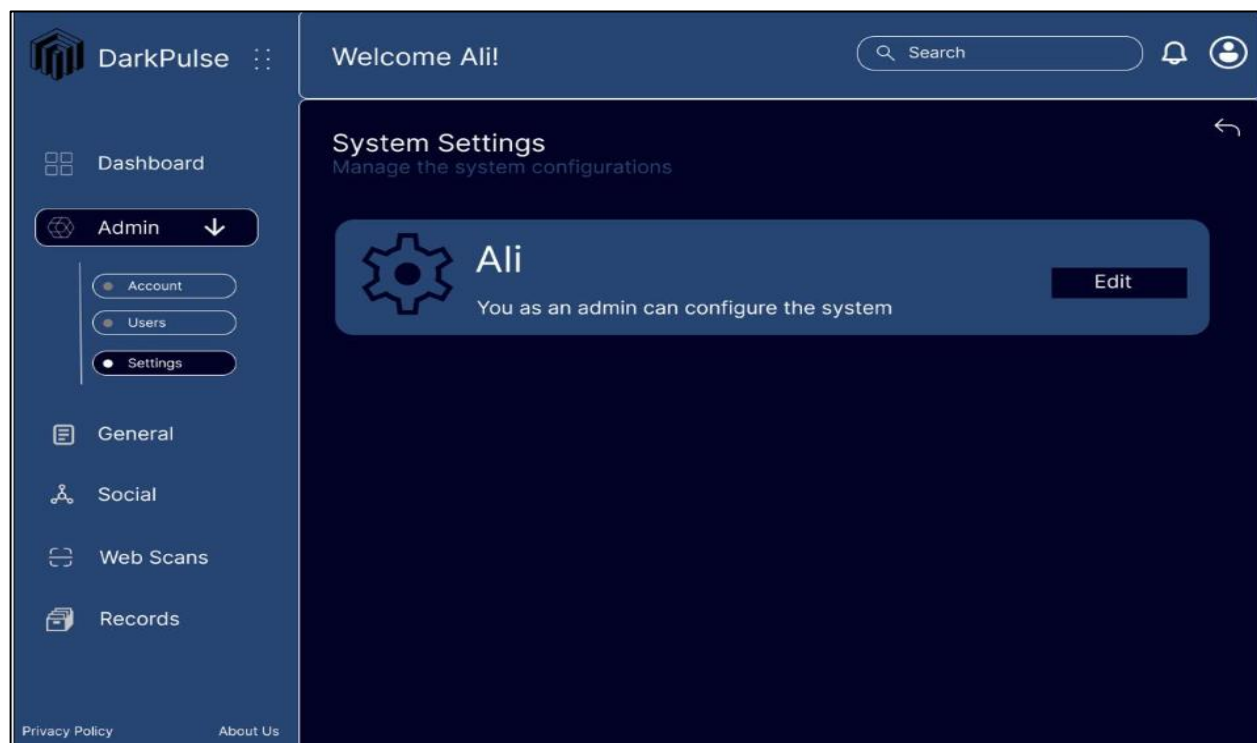
4.2 Screens



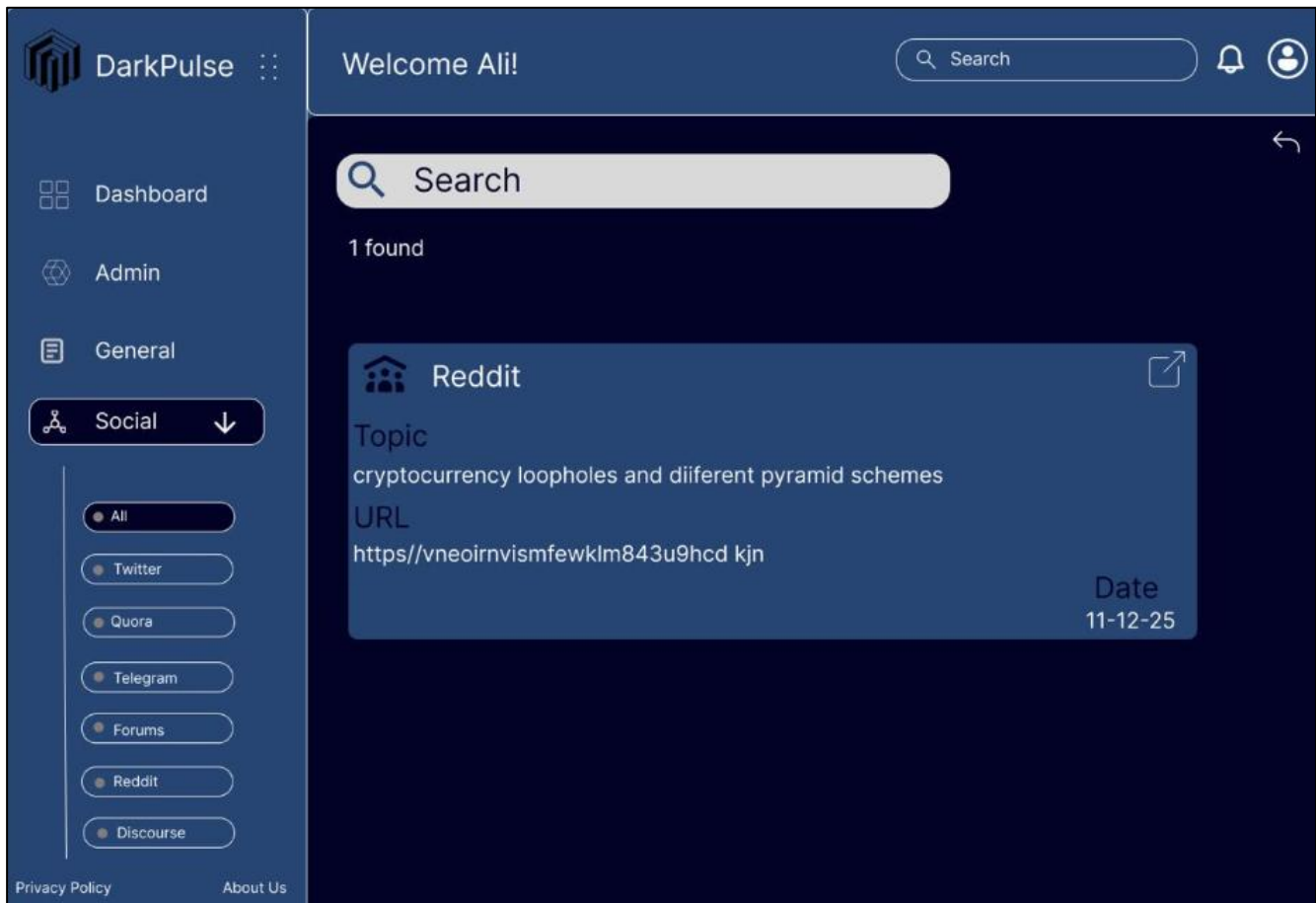
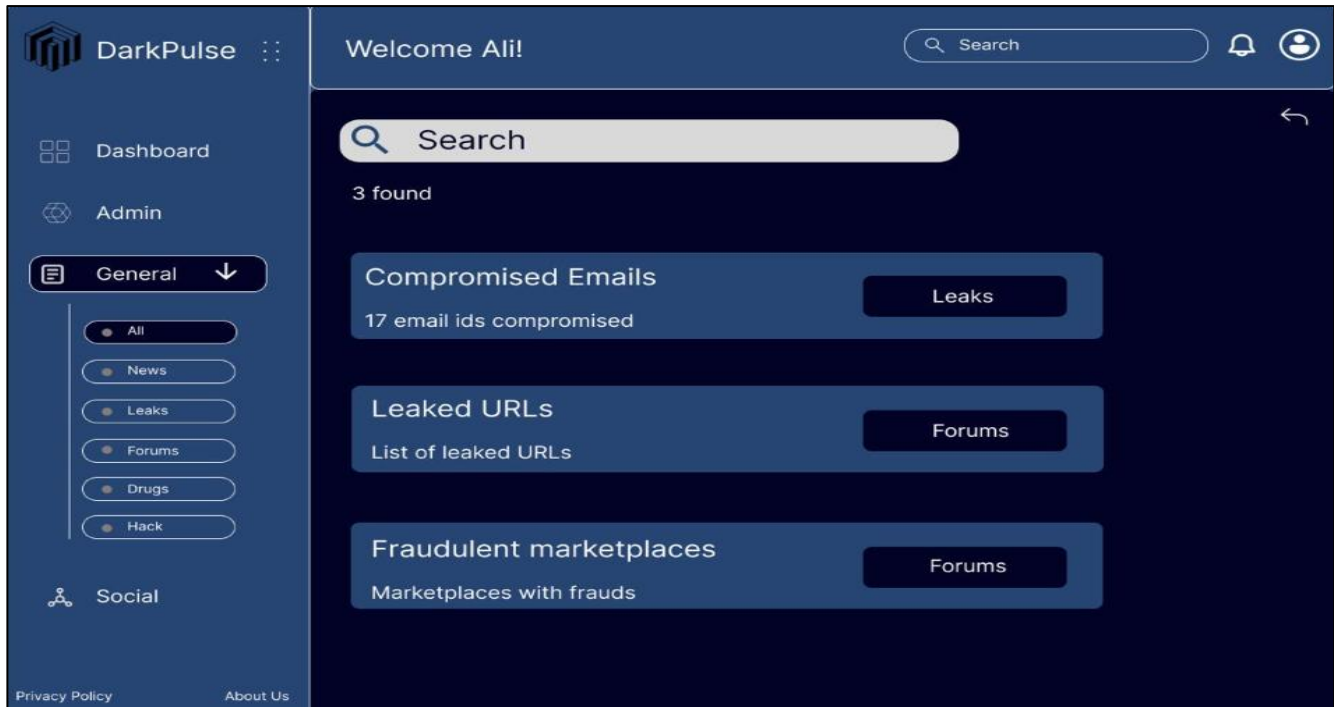


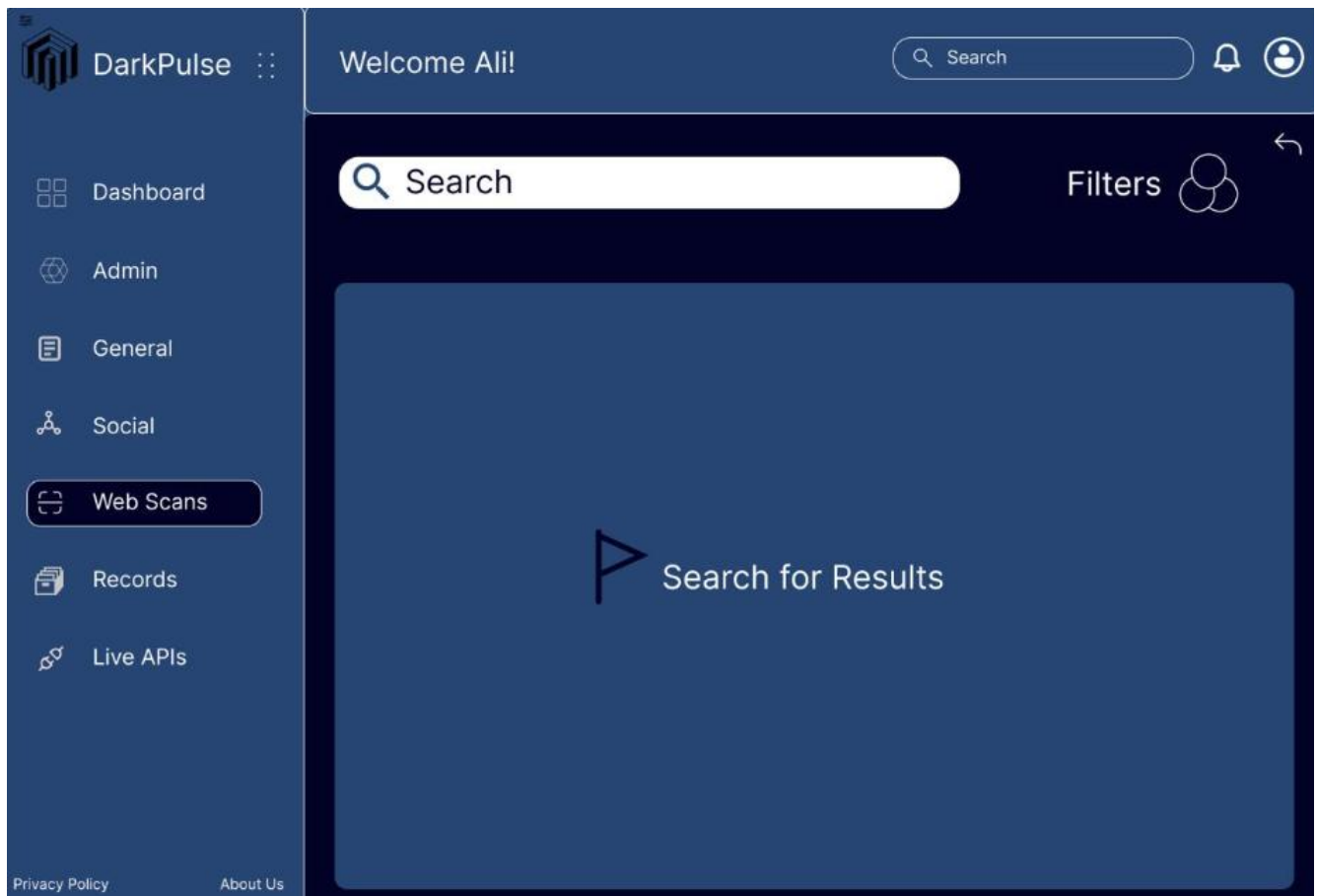
The screenshot shows the 'Users' management page in the DarkPulse admin interface. The left sidebar contains navigation links: Dashboard, Admin (selected), Account, Users, Settings, General, Social, Web Scans, and Records. The main content area displays a table of users with columns for Sr#, Name, Email, Role, and Action. The table lists five users: Ali (admin), Abdullah (admin), Omar (member), Isa (member), and Test (test). Each user has an edit icon in the Action column. The top header shows 'Welcome Ali!' and a search bar.

Sr#	Name	Email	Role	Action
1	Ali	ali@gmai...	admin	
2	Abdullah	abdullah...	admin	
3	Omar	omar@gma...	member	
4	Isa	isa@gmai..	member	
5	Test	-	test	



The screenshot shows the 'System Settings' page in the DarkPulse admin interface. The left sidebar is identical to the previous page, with 'Settings' now selected under the 'Admin' menu. The main content area displays the 'System Settings' title and subtitle 'Manage the system configurations'. Below this, there is a section for the user 'Ali' with a gear icon and the text 'You as an admin can configure the system'. An 'Edit' button is visible next to the user name.





4.3 Additional Information

Strong security measures are mandatory. These include role-based access control, encrypted database storage, secure authentication, protected API endpoints and detailed activity logging. All communication between the client and server must be encrypted.

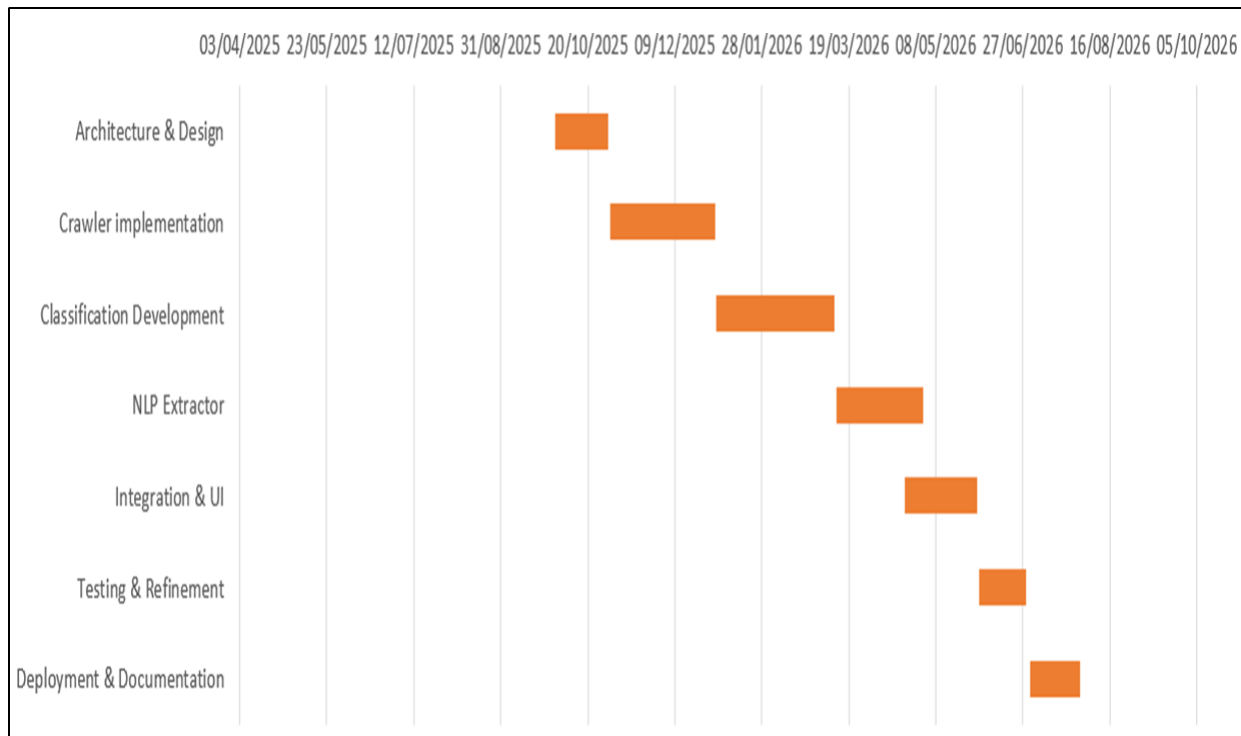
[5] Other Design Details

Tor Network Interface & Latency Management

The dependency on the Tor network introduces unique design constraints related to latency and stability. The system implements specific **Circuit Management** protocols to handle these issues.

- **Circuit Rotation:** To prevent blocking and ensure anonymity, the Crawler Controller interacts with the Tor Control Port (Port 9051) to request a NEWNYM signal (New Identity) every 10 minutes or after 50 consecutive requests.
- **Timeout Handling:** Unlike standard web crawlers, the design accounts for high latency. The request timeout is set dynamically to **30 seconds** (vs. standard 5s) to accommodate the multi-hop onion routing.
- **Rate Limiting:** To avoid destabilizing the hidden services being scanned, the crawler enforces a strict **politeness delay** (e.g., 2 seconds between requests to the same domain).

[6] Revised Project Plan



[7] References

[1] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python. Sebastopol, CA: O'Reilly Media, 2009.

[2] R. Mitchell, Web Scraping with Python: Collecting More Data from the Modern Web. Sebastopol, CA: O'Reilly Media, 2018.

[3] N. Fedorova and A. Monakhova, "A framework for automated detection of data breaches in underground markets," Journal of Cybersecurity, vol. 7, no. 1, 2021.

[4] A. Singhal, "Introduction to web scraping: Tools, techniques and applications," Int. J. Computer Applications, vol. 176, no. 32, pp. 1–6, 2020.

[5] G. Owen and N. Savage, "The Tor Dark Net," Global Commission on Internet Governance, Paper Series No. 20, 2016.

Appendix A: Glossary

Terms:

Dark Web: Hidden network accessed through TOR.

Crawler: Tool that automatically collects data from web pages.

Onion Site: Website ending in .onion on the TOR network.

Data Leak: Exposed sensitive details found online.

Preprocessing: Cleaning text before analysis.

Embedding: Numeric representation of text created by BERT.

Classification: AI assigning a category to text (leak/threat/normal).

Confidence Score: How sure the model is about its prediction.

Result Storage: Saving processed data in the database.

Acronyms:

AI: Artificial Intelligence

NLP: Natural Language Processing

ML: Machine Learning

DL: Deep Learning

URL: Uniform Resource Locator

LR: Logistic Regression

API: Application Programming Interface

UI: User Interface

DB: Database

SQL: Structured Query Language

TOR: The Onion Router (Anonymous Network)

Appendix B: IV & V Report

(Independent verification & validation)

IV & V Resource

Name

Signature

S#	Defect Description	Origin Stage	Status	Fix Time	
				Hours	Minutes
1					
2					
3					
...					

Table 1: List of non-trivial defects

This document has been adapted from the following:

1. Previous project templates at UCP
2. High-level Technical Design, Centers for Medicare & Medicaid Services. (www.cms.gov)